1. Data Organization Design

1.1 Architecture Overview

FreightLens ingests data from three sources with fundamentally different reliability profiles: a high-trust core system, a medium-trust AI extraction agent (5-8% error rate), and third-party external feeds with variable freshness. The central challenge is unifying these into a single, agent-queryable surface while ensuring bad data from the AI extraction layer cannot silently corrupt recommendations built on core system data.

I chose a **three-layer Medallion Architecture (Bronze → Silver → Gold)** on ClickHouse. Each layer has a distinct contract: Bronze stores everything unmodified, Silver applies quality and enrichment logic, and Gold exposes only business-ready views with freshness guarantees that agents query directly.

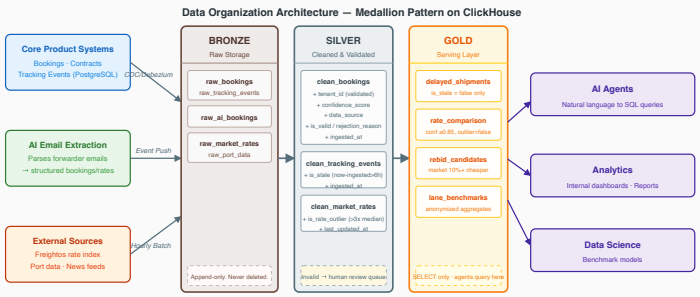

Figure 1: Data Organization Architecture — 3 sources through Bronze/Silver/Gold layers to agent consumption

1.2 Ingestion Paths

Source	What it brings	Ingestion method	Trust level
Core product systems	Authoritative structured data: bookings, contracts, tracking events from the FreightLens PostgreSQL application database	CDC via Debezium watching the PostgreSQL WAL log. Seconds-level latency with no additional query load on the production database.	High — source of truth
AI email extraction agent	Spot rate offers and booking confirmations parsed from forwarder emails into structured JSON	Event push — the agent writes one record per parsed email to Bronze. No streaming infrastructure needed for discrete per-email events.	Medium — 5-8% error rate
External sources	Market rate benchmarks (Freightos), port congestion alerts, carrier schedule updates	Hourly batch API poll with idempotent upsert. Sufficient — freight market rates do not change meaningfully faster than hourly.	Medium — third-party availability varies

1.3 Stage Breakdown

Bronze — Raw Storage

Append-only. Every record from every source is written here unchanged. No transformations, no deletions. If Silver validation logic changes, Bronze provides the clean replay substrate.

Attribute	Detail
Tables	raw_bookings, raw_tracking_events, raw_contracts, raw_ai_extracted_bookings, raw_market_rates, raw_port_data
Added metadata	source_tag, arrived_at timestamp, system-generated raw_id. Nothing else.
Rejected records	Nothing is rejected at Bronze. Completely malformed payloads are stored with a parse_error flag — never silently dropped.

Silver — Cleaned & Validated

All data quality logic lives here. Rejected records stay in Silver with a populated rejection_reason — never silently discarded. AI-extracted records below the confidence threshold enter a human review queue.

Validation / Enrichment	Rule applied
tenant_id check	Must be present and match a known tenant in the registry. Missing → is_valid = false.
Rate outlier check	Rate must be within 0.1×–10× the rolling median for that lane (from core_system records only). Outside range → is_rate_outlier = true → does NOT promote to Gold.
Date logic	ETD must precede ETA. Both must be valid calendar values.
Deduplication	On (booking_id, data_source) pair — CDC can deliver the same event twice during recovery.
confidence_score	1.0 for core_system (authoritative). AI model's own confidence for email extractions. 0.95 for external API records.
is_stale	Computed for tracking events: true if (now() – ingested_at) > 6 hours. Primary guard against the ghost-delay failure mode.
human_verified	Defaults to false for all AI-extracted records. Flipped to true via the ops review queue after manual confirmation.

Gold — Serving Layer

Business-ready views that agents query directly. Freshness and confidence guarantees are structural — not optional filters agents might forget to apply.

View	Contains	Key constraint
delayed_shipments	Shipments where latest tracking event indicates delay	is_stale = false enforced in view definition
rate_comparison	Contract vs. market vs. spot rates per shipment	confidence_score ≥0.85 AND is_rate_outlier = false for all rates
rebid_candidates	Shipments where a verified lower rate exists on the lane	Market rate ≥10% cheaper than contract; inherits rate_comparison constraints
lane_benchmarks	Anonymized aggregate rates per lane across all standard-tier tenants	No tenant_id. Only median, p25, p75, client_count. Minimum client_count = 5 (privacy floor). Refreshed nightly.

1.4 Key Schema Sketches

```
-- Silver: clean_bookings
booking_id          VARCHAR      -- From source system
tenant_id           VARCHAR      -- Critical: enforced at every layer
origin_port         VARCHAR
destination_port    VARCHAR
carrier             VARCHAR
contract_rate       DECIMAL(12,2)
etd                 DATETIME     -- Expected Time of Departure
eta                 DATETIME     -- Expected Time of Arrival
status              ENUM('active','delayed','completed','cancelled')
data_source         ENUM('core_system','ai_email','external_api')
confidence_score    FLOAT        -- 1.0 for core_system; AI model score for email
is_rate_outlier     BOOLEAN      -- True if rate deviates >3x lane median
human_verified      BOOLEAN      -- False by default for AI-extracted records
is_valid            BOOLEAN      -- Passed all Silver validations?
```

```
rejection_reason    VARCHAR      -- Populated if is_valid = false
ingested_at         DATETIME     -- When WE received this record (for freshness)
event_time          DATETIME     -- When the booking occurred at source
```

```
-- Silver: clean_tracking_events
event_id            VARCHAR
booking_id          VARCHAR      -- FK to clean_bookings
tenant_id           VARCHAR      -- Enables tenant-scoped queries without JOIN
event_type          ENUM('departed','delayed','arrived','customs_hold','port_congestion')
new_etd             DATETIME     -- Populated when event_type = 'delayed'
event_time          DATETIME     -- When the event occurred at carrier/port
ingested_at         DATETIME     -- When WE received it (different from event_time)
is_stale            BOOLEAN      -- Computed: (now() - ingested_at) > 6 hours
data_source         VARCHAR
```

1.5 Architecture Justification

Alternative considered	Why ruled out
Two-layer model (raw + serving only)	AI-extracted data at 5-8% error rate needs an intermediate quarantine and confidence-scoring stage. Merging this directly into a serving layer creates fragile, hard-to-maintain views with complex conditional logic that is difficult to test and reason about.
Lambda architecture (parallel batch + streaming)	Two parallel pipelines maintaining consistency is operationally expensive for a 4-person team. CDC at seconds-level latency eliminates the streaming need without the operational burden.
Agent queries directly on OLTP (PostgreSQL)	Analytical queries scanning millions of rows degrade production app performance. Non-negotiable separation required.

2. Access Architecture

2.1 All System Participants and Their Access Paths

Participant	Access path	Permitted layer	Constraint
Client users (ops, managers)	Via AI agent only — no direct DB access	Gold (via agent)	Scoped to own tenant_id at middleware layer
AI agents	Query middleware → ClickHouse Gold	Gold only	SELECT only; tenant_id auto-injected; no Silver or Bronze access
Data Science / Analytics	Read-only ClickHouse credentials (internal)	Silver + Gold	Tenant-scoped credentials; cross-tenant only via lane_benchmarks
CDC pipeline (Debezium)	Dedicated writer service account	Bronze (write only)	Cannot read; write-only
AI extraction agent	Writer API endpoint	Bronze (write only)	Cannot read other tenants' Bronze data
Silver processing pipeline	Scheduled job (service account)	Bronze (read), Silver (write)	Cannot access Gold or production OLTP
FreightLens ops team	Human review queue UI	Silver (targeted update)	Can only update human_verified flag on flagged rows

2.2 Tenant Isolation Strategy

Every Silver and Gold table contains a **tenant_id column**. Isolation is enforced by a Query Middleware layer — a structural guarantee, not a coding convention that agents might accidentally violate.

How the middleware works

1. Extracts `tenant_id` from the authenticated user's JWT token
2. Parses the generated SQL into an abstract syntax tree (AST)
3. Injects `AND tenant_id = '{tenant_id}'` into the WHERE clause of every table reference
4. Validates the injection succeeded before dispatching to ClickHouse

```
-- Agent generates (no tenant filter):
SELECT * FROM delayed_shipments WHERE week = current_week()

-- Middleware transforms to:
SELECT * FROM delayed_shipments
WHERE week = current_week()
  AND tenant_id = 'tata_motors'    -- injected automatically from JWT
```

> **Fallback behavior:** If the middleware cannot resolve a tenant_id (JWT missing, expired, or malformed), the query is *rejected immediately* — never reaches ClickHouse. The agent returns: "Session expired, please re-authenticate." No silent data leakage is possible in this failure path.

2.3 Dedicated Database for Enterprise Tier

The Sales requirement for a fully dedicated ClickHouse instance is handled via a **Connection Router** — a transparent abstraction requiring zero changes to agent code.

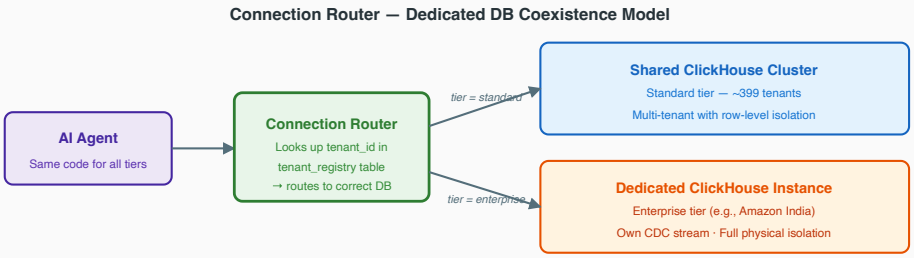

Figure 2: Connection Router — same agent code, different ClickHouse target per tenant tier

```
tenant_registry:
tenant_id     | tier        | clickhouse_host                | db_name
tata_motors   | standard    | shared-ch.freightlens:9000     | freightlens_prod
reliance      | standard    | shared-ch.freightlens:9000     | freightlens_prod
amazon_india  | enterprise  | amazon-ch.freightlens:9000     | amazon_fl_prod
```

The enterprise client's ClickHouse instance receives its own dedicated CDC stream from Debezium. It is excluded from the nightly lane_benchmarks aggregation job per their isolation requirement (agreed as part of the enterprise tier contract — they opt out of cross-client benchmarks).

2.4 Cross-Client Benchmarks Without Violating Isolation

Data Science wants: "You pay 15% above market on Chennai-Hamburg." This requires aggregating across tenants. A nightly anonymization job aggregates rates per lane across all standard-tier tenants — no tenant identifiers in the output. Individual client rates are never exposed to another tenant.

Privacy boundary: lane_benchmarks stores only: lane, median_rate, p25_rate, p75_rate, and client_count. client_count minimum threshold = 5. Lanes with fewer than 5 clients are suppressed to prevent reverse-engineering individual rates from aggregates.

2.5 NL → SQL Safety Pipeline

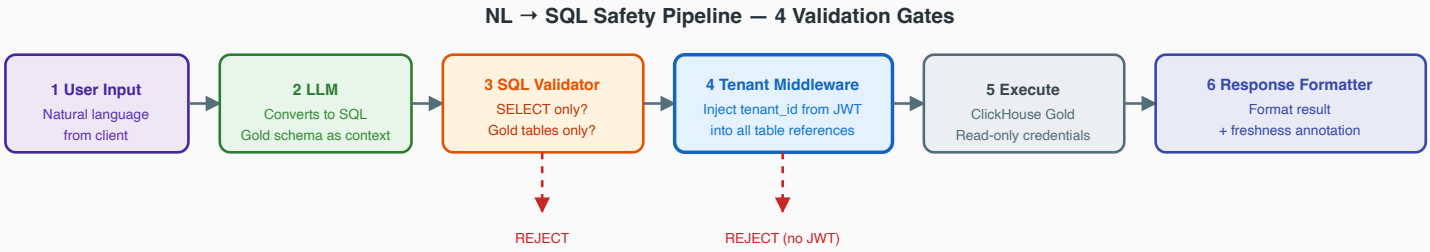

Figure 3: NL → SQL pipeline — query passes 4 validation gates before reaching ClickHouse

Step	Component	What happens	Rejection condition
1	User	Types natural language query in FreightLens UI	—
2	LLM (SQL Generator)	Converts NL to SQL using Gold schema as system prompt context	—
3	SQL Validator	Parses SQL AST; checks statement type and table references	Non-SELECT statement; reference to Bronze/Silver tables; unknown columns
4	Tenant Middleware	Extracts tenant_id from JWT; injects filter into all table WHERE clauses	JWT missing, expired, or tenant_id unresolvable
5	ClickHouse Gold	Executes query with read-only service account; 30-second timeout enforced	Query timeout
6	Response Formatter	Agent formats result and appends freshness annotation ("data as of X hours ago") — mandatory on all responses involving tracking data	—

3. End-to-End Trace

Scenario: Rajesh, a logistics manager at Tata Motors, asks the FreightLens agent: *"Which of our shipments have delayed ETDs this week? Should we consider rebidding any lanes?"*

Step	Component	What happens	Data access / constraint
1	Rajesh (UI)	Types the question in FreightLens UI	—
2	Agent	Resolves user identity from JWT → tenant_id = 'tata_motors'	Auth service. Fails immediately if JWT invalid.
3	NL→SQL (LLM)	Generates: SELECT * FROM delayed_shipments WHERE week = current_week()	Gold schema injected as system prompt context
4	SQL Validator	Checks: SELECT only ✓ · Gold tables only ✓ · valid columns ✓	Rejects on any failure. Query never reaches ClickHouse.
5	Tenant Middleware	Injects AND tenant_id = 'tata_motors' into the query	Tata Motors cannot see other tenants' data
6	ClickHouse Gold	delayed_shipments view filters: is_stale = false AND tenant_id = 'tata_motors'. Returns SH-001 (ETD delayed 4 days; tracking last updated 2 hours ago ✓)	Freshness constraint is in the view definition — agents cannot bypass it
7	Agent → ClickHouse	Queries rate_comparison for SH-001's lane (Chennai→Hamburg). Tenant filter auto-injected again.	Only rates with confidence ≥0.85 and is_rate_outlier = false are included
8	ClickHouse returns	contract_rate = Rs2.8L, market_rate = Rs2.2L (Freightos, 4hrs old ✓), spot_offer = Rs2.1L (AI email, confidence 0.92 ✓)	—
9	Agent (business logic)	Saving = (Rs2.8L – Rs2.1L) × 10 containers = Rs7L. Spot offer valid until Aug 31 ✓	Business logic in agent layer, not SQL
10	Agent → Rajesh	"SH-001 is delayed 4 days (tracking data as of 2 hours ago). Spot offer Rs2.1L vs. your contract Rs2.8L — potential saving of Rs7L. Offer valid until Aug 31. Shall I initiate a rebid?"	Freshness annotation is mandatory in every response involving tracking data
11	Rajesh approves → system	Agent creates a rebid ticket via a separate write API — not through ClickHouse (agents have SELECT-only access to the warehouse)	Write actions route through the OLTP system, not the data warehouse

How tracking data freshness is surfaced

Every agent response that involves tracking data includes an explicit annotation ("tracking data as of X hours ago"). This is enforced at the Response Formatter layer — not optional. If tracking data is stale (is_stale = true), the `delayed_shipments` view excludes that shipment. The agent instead returns: "I found a shipment with a delayed status but the tracking data is [N] hours old. Please verify with the carrier directly before acting."

4. Failure Analysis

FAILURE MODE 1

Tracking Event Silence — The "Ghost Delay"

Scenario: The carrier's tracking API stops sending events for 18 hours (API outage or rate limit). SH-001's last recorded event says "ETD delayed to Sep 2." The ship actually departed on Sep 2 — but the "departed" event was never received. The database still shows "delayed."

Root cause of silence: The failure is an *absence of events*, not a system error. Debezium is running. ClickHouse is healthy. Silver processing completes without errors. Standard monitoring watches for pipeline failures — but there are none here. A tracking event that was never sent produces no error, no dead-letter queue entry, nothing. The only signal is the staleness of existing data, which standard alerting does not measure.

Impact radius:

- `delayed_shipments` Gold view: SH-001 incorrectly appears as delayed
- Agent recommendation: Initiates unnecessary rebid discussions with a carrier for a shipment already at sea
- Operations escalation and client trust damage (this exact scenario occurred per the brief)

Mitigation in this design:

- **is_stale flag (primary guard):** clean_tracking_events marks any record stale if (now() – ingested_at) > 6 hours. The delayed_shipments Gold view structurally excludes is_stale = true records. The ghost delay cannot reach an agent recommendation.
- **Explicit stale messaging:** When SH-001 is excluded due to staleness, the agent responds: "I found a shipment with a delayed status but the tracking data is [N] hours old. Please verify directly with the carrier before acting."
- **Proactive monitoring alert:** If any active tenant produces zero new tracking events for >8 consecutive hours → page on-call. Detects carrier API silence before any agent query is affected.

FAILURE MODE 2

AI Extraction Error Silently Poisoning Rate Analytics

Scenario: The email extraction agent misreads "Rs2.1L" as "Rs21L" — a 10x error from a decimal parsing failure. Confidence score is 0.88 (above the 0.85 threshold) because the email was cleanly formatted and all other fields were correctly extracted. The is_rate_outlier check should catch this — but on a new lane with sparse core_system records, the lane median may be unreliable. The bad record promotes to Gold.

Root cause of silence: The record has a valid numeric rate, a valid lane, a valid expiry date, and a confidence score above threshold. It passes all schema validations — no null, no wrong type, no constraint violation. Standard data quality monitoring catches missing or malformed data; it does not catch plausible-but-wrong values. The record is indistinguishable from a legitimate booking.

Impact radius:

- `rate_comparison`: inflated spot rates make the contract rate appear comparatively cheap
- `rebid_candidates`: real rebid opportunities missed because the comparison baseline is artificially elevated
- `lane_benchmarks`: cross-client aggregates on the affected lane skewed upward
- Data Science model outputs: "you're paying below market" when the client is actually overpaying

Mitigation in this design:

- **Rate outlier check (Silver):** If AI-extracted rate > 3× the rolling median for that lane (core_system records only), is_rate_outlier = true. Does NOT promote to Gold. Enters the human review queue regardless of confidence score.
- **Dual-threshold requirement (Gold):** AI-extracted rates must satisfy both: confidence ≥0.85 AND is_rate_outlier = false. Reduces single-signal failure mode.
- **Weekly reconciliation job:** Compares AI-extracted rates against confirmed core_system records for the same booking. Tracks per-forwarder, per-email-domain extraction accuracy to identify systematic failures early.
- **New lane handling:** For lanes with fewer than 10 core_system records (sparse median), all AI-extracted rates route to human review regardless of confidence. The outlier check is only reliable with sufficient reference data.

5. Tradeoffs

5.1 Three Deliberate Omissions

What I left out	Chosen instead	Functional cost	Trigger to revisit
Real-time streaming (Kafka + Flink)	CDC via Debezium (~seconds for core system); hourly batch for external APIs	External market rate data can be up to 1 hour stale. Cannot support sub-30-second latency for live auction rebids or real-time carrier rate feeds.	A client SLA requiring <30-second tracking freshness, or a product feature triggering automated rebids within seconds of a rate change.
Vector embeddings / semantic search	SQL-only queries on structured Gold views	Agents cannot answer questions like "find shipments similar to ones that had port delays last quarter." All agent queries must be expressible as structured SQL filters.	Agent roadmap includes "find similar historical incidents" or "explain why this shipment is anomalous" — use cases requiring semantic similarity.
Full GDPR right-to-erasure in Bronze	Deletion honored at Silver and Gold only; Bronze is append-only	Bronze retains PII in raw JSON payloads even after a client's deletion request is honored at Silver/Gold. Creates compliance exposure for EU clients under active GDPR enforcement.	First enterprise EU client subject to GDPR enforcement, or legal requirement for full deletion including raw logs (would require crypto-shredding approach on Bronze).

5.2 First Bottleneck at 3× Current Volume

At 3× volume (~1,200 clients, ~30M tracking events/day), the first bottleneck will be **Gold layer views computed on-demand at query time**. Views like `delayed_shipments` and `rate_comparison` currently JOIN Silver tables live on every agent query. At 3× agent query volume with concurrent sessions across 1,200 clients, these joins will degrade significantly.

Required architectural shift: Materialize the Gold views into pre-computed tables, refreshed every 15 minutes by a background job. This moves latency from query-time (unbounded) to refresh-time (controlled). Agents query pre-computed results with guaranteed freshness annotations. The 15-minute staleness window is acceptable for freight — clients are not making sub-minute decisions on cargo that is mid-ocean.

The second bottleneck is the single shared ClickHouse cluster serving both agent queries and Silver processing. Adding read replicas offloads agent query traffic with no architectural redesign required.

5.3 Critical Self-Assessment — Most Controversial Design Choice

My most contentious decision is **allowing AI-extracted data into the Gold serving layer** (subject to confidence and outlier thresholds) rather than quarantining all AI-extracted data until human verification.

The case against this choice: At 5-8% error rate, even a 0.88 confidence record has a non-trivial probability of being wrong. A strict position would quarantine all AI-extracted data until a human approves it — zero automated promotion.

Why I made this choice: Freight spot rates from forwarders are time-sensitive and come almost exclusively via email. A 48-hour spot offer reviewed by a human queue with a 24-hour SLA is worthless by the time it reaches the agent. Quarantining all AI data removes the email extraction feature from production value. At 92-95% extraction accuracy, the dual-threshold guard (confidence + outlier check) catches the most dangerous errors without blocking the majority of correct records.

What would cause me to change this decision: If the error rate climbs above 10%; if a single AI-extracted rate causes a client to execute a rebid at the wrong price (a material financial loss, not just a bad recommendation); or if the weekly reconciliation job reveals systematic extraction failures on specific forwarder email domains. At that point I would shift to full quarantine with an expedited 2-hour human review SLA and work with the product team on replacing free-text email with a structured rate submission API.